

4x05 — Reconstrução de Ataque

Campanha HEALTHBANE: Guia de Estudos

Um guia de preparação, tarefa por tarefa, para as Tasks 0, 1, 2, 3, 4, 5, 7, 8, 9, 12 e 15. Este guia explica o que cada exercício está pedindo, qual habilidade ele quer desenvolver, e como abordá-lo — incluindo padrões específicos de *jq* / *grep* / *date* / *diff* para usar. Ele não resolve os exercícios: os IOCs, timestamps e achados reais dos seus arquivos de evidência são seus para descobrir.

Antes de Começar: O Que Este Projeto Realmente É

Todo projeto deste curso até agora investigou UM domínio de evidência por vez: o 4x00 leu emails, o 4x01 leu capturas de pacotes, o 4x02 leu relatórios de inteligência, o 4x03 leu amostras de malware, o 4x04 leu logs de SIEM. Cada um ensinou você a ser rigoroso *dentro* de um único tipo de evidência. O 4x05 é diferente em natureza, não só em escopo: ele entrega os resultados de todas as cinco investigações anteriores mais quatro arquivos novos de evidência de resposta a incidente (captura de memória, imagem de disco, 14 dias de logs de firewall, e as anotações brutas do time de IR) e pede para você produzir UM relato coerente, citado e cronológico do ataque inteiro.

A habilidade sendo testada não é “você sabe rodar jq contra um arquivo novo.” É **correlação entre evidências**: você consegue pegar uma afirmação de uma fonte, encontrar (ou não encontrar) corroboração para ela em uma fonte completamente diferente, e raciocinar com honestidade sobre o que essa concordância ou discordância significa? Essa habilidade é o que separa um analista júnior que produz uma pilha de achados de um analista sênior que produz entendimento.

Quatro ideias que você precisa antes de escrever uma linha de código

- **CONFIRMED / PROBABLE / POSSIBLE.** Isso não é uma escala de “sensação”, é uma regra: CONFIRMED significa evidência direta de *pelo menos duas fontes independentes*. PROBABLE significa evidência forte de uma fonte mais contexto de apoio (por exemplo, o horário bate com um padrão já conhecido). POSSIBLE significa que o playbook conhecido do atacante torna isso plausível, mas você não consegue apontar um artefato específico que prove que aconteceu. Toda afirmação que você escrever nas Tasks 4, 5 e 7 precisa de um desses três rótulos, e você precisa conseguir defender por que escolheu aquele e não um nível mais alto.
- **CONVERGED / SINGLE-SOURCE / CONFLICTED.** É a versão desse mesmo conceito no nível de IOC/evidência (usada na Task 4). Um indicador que aparece em dois métodos de coleta não relacionados (digamos, um achado derivado de PCAP do 4x01 E um log de firewall do IR) é uma evidência muito mais forte do que o mesmo indicador aparecendo uma vez só. Quando duas fontes discordam abertamente, isso é CONFLICTED e precisa ser resolvido, não ignorado por média.
- **Âncoras temporais.** Nem todo timestamp na sua evidência é igualmente confiável. Uma âncora temporal é um timestamp no qual você tem alta confiança (por exemplo, o campo Received: de um cabeçalho de email, ou um horário de criação no \$MFT do disco) que você usa para checar ou ordenar tudo o mais. Quando duas fontes discordam sobre um horário, não escolha o número mais “bonito” — descubra qual método de coleta está mais próximo do evento real (ponto de captura de rede vs. atraso de agregação de log vs. desvio de relógio) e diga isso explicitamente.
- **Citações não são decoração opcional.** Os requisitos do projeto são explícitos: arquivo-fonte, timestamp, host, conta de usuário, para toda afirmação. Uma afirmação sem citação não é um achado — é especulação. Crie esse hábito já na Task 0: anote exatamente de qual arquivo e qual campo você tirou cada fato — isso vai te poupar muito trabalho quando você chegar nas Tasks 5 e 7, onde você estará citando cinco ou seis fontes por parágrafo.

Seu kit de ferramentas, e quando você vai realmente usar cada uma

Ferramenta	Para que serve aqui
jq	Todo arquivo de evidência em JSON (sessões de firewall, mapeamento ATT&CK do 4x02, IOC master, o layer de 80% de cobertura) é consultado com jq. Conheça <code>select()</code> , <code>test()</code> para regex, <code>group_by()</code> , e as funções de data (<code>strptime</code> / <code>mktime</code> / <code>strftime</code>) para cálculo com timestamps.

grep -E / -i	Extrair campos estruturados dos relatórios em texto puro (memory_artifacts.txt, disk_forensics_report.txt, os resumos em previous_findings/*.txt). Esses são texto semi-estruturado, não JSON — grep com boa regex é sua ferramenta principal aqui.
sort / uniq -c	Construir as contagens de frequência e tabelas de “top N” que as saídas esperadas pedem (destinos externos principais por bytes, entradas de prefetch mais executadas, etc.).
date -d ... +%s	Converter timestamps para segundos-epoch para fazer aritmética com eles — é assim que você calcula os deltas de desvio de relógio e as janelas de dwell-time que as Tasks 4, 5 e 7 pedem explicitamente. Lembre da lição de fuso horário do 4x04: sempre saiba em qual fuso horário cada fonte realmente registra seus timestamps antes de compará-los.
diff	Comparar diretamente duas listas derivadas — por exemplo, a lista de IOCs implicada pelo mapeamento ATT&CK do 4x02 contra a lista de IOCs em reference/healthbane_ioc_master.json — para revelar mecanicamente o que mudou, em vez de comparar dois arquivos longos “a olho”.
wc -l	Contagens rápidas de sanidade (quantas sessões, quantos arquivos recuperados) antes de confiar na saída de uma consulta mais complexa.

O que é este exercício

Antes de analisar um único artefato, você cataloga toda a paisagem de evidências: cada arquivo em `ir_evidence/`, `previous_findings/` e `reference/`, que tipo de evidência cada um é, que período de tempo cobre, e quão confiável você deve considerá-lo. Isso deliberadamente **NÃO** é análise — é reconhecimento da sua própria base de evidências, a mesma disciplina do `3-data_recon.sh` lá no 4x04, só que um nível acima: em vez de perfilar um único export de SIEM, você está perfilando onze fontes diferentes, de tipos e confiabilidades bem distintas.

O que você deve aprender

- Por que começar pela análise antes do inventário produz **viés de disponibilidade** — você acaba dando peso demais à evidência que por acaso abriu primeiro.
- Como atribuir um nível de confiabilidade (HIGH / MEDIUM / LOW) com honestidade: evidência bruta de IR (memória, disco, logs de firewall) coletada sob cadeia de custódia controlada é HIGH; o *resumo* que um projeto anterior fez da sua própria evidência bruta é MEDIUM (você está confiando na interpretação de outra pessoa, não nos pacotes/logs originais); as próprias anotações preliminares do time de IR são explicitamente LOW até você pessoalmente verificá-las contra a evidência mais sólida.
- Como uma matriz de cobertura (tipo de evidência × período de tempo) transforma uma sensação vaga de “provavelmente temos lacunas” em uma lista específica e defensável de lacunas — que vira sua seção de “perguntas críticas”.

Como abordar

- Não escreva isso como um grande pipeline de `jq/grep`. Essa tarefa é principalmente sobre ler com cuidado e transcrever fatos estruturados — trate cada arquivo-fonte como uma testemunha que você entrevista uma vez: anote nome / tipo / intervalo de datas / confiabilidade / conteúdo-chave para cada um, e siga em frente.
- Para as fontes em texto puro, procure por qualquer coisa que pareça uma data, um intervalo de datas, ou uma declaração de “período coberto” perto do topo do arquivo — resumos de investigação costumam declarar seu próprio escopo explicitamente.
- Para as fontes JSON (logs de firewall, IOC master, layer ATT&CK), extraia o timestamp mínimo/máximo real com `jq` em vez de adivinhar pelo nome do arquivo ou por uma descrição vaga.
- Monte a matriz de cobertura **SÓ DEPOIS** de ter as onze entradas catalogadas — coloque os intervalos de data lado a lado e as lacunas vão ficar visualmente óbvias, exatamente como no gráfico ASCII do exemplo de saída esperada.

Comandos e padrões para usar

```
# Pegar o timestamp mínimo/máximo real de um arquivo JSON (não chute):
jq -r '.[].timestamp' ir_evidence/firewall_sessions_ws_recv_03.json | sort | head -1
jq -r '.[].timestamp' ir_evidence/firewall_sessions_ws_recv_03.json | sort | tail -1

# Extrair uma linha de data / escopo declarada de um resumo em texto puro:
grep -iE 'date|period|coverage|window|scope' previous_findings/4x00_phishing_summary.txt

# Contar entradas de um array JSON rapidamente, como checagem antes de confiar num arquivo:
jq 'length' reference/healthbane_ioc_master.json
```

Fique de olho em

Resista à tentação de escrever sua seção de “perguntas críticas” de forma genérica (“que mais a gente não sabe?”). Cada pergunta deve remontar a uma lacuna específica que você identificou na matriz de cobertura — um período sem evidência, ou uma fase do ataque com apenas uma fonte de apoio. Se você não conseguir apontar a lacuna que gerou a pergunta, esse inventário ainda não te deu direito a essa pergunta.

O que é este exercício

Você está processando um único arquivo — `ir_evidence/memory_artifacts.txt` — que contém uma foto, num momento específico, da RAM do WS-RECV-03: a lista de processos no momento da captura, conexões de rede ativas, DLLs/módulos carregados, e extratos do registro (incluindo tarefas agendadas). O gancho conceitual que a tarefa te dá é importante: memória mostra o que estava *rodando*, o que é um tipo de evidência fundamentalmente diferente de disco (o que foi *armazenado*) ou captura de rede (o que foi *transmitido*). Um processo que já apagou o próprio binário do disco ainda está presente neste arquivo.

O que você deve aprender

- Por que a forense de memória captura coisas que disco e rede estruturalmente não conseguem — e, ao contrário, por que memória é incompleta (é uma foto de UM momento, não um histórico).
- Como fazer a triagem de status de IOC: para cada artefato encontrado, ele é KNOWN (já está em `reference/healthbane_ioc_master.json` de um projeto anterior), NEW (nunca visto antes), ou MODIFIED (uma variante — por exemplo, mesma família de ferramenta, nome de arquivo ou hash diferente)? É a mesma disciplina que o 4x02 ensinou para indicadores de inteligência, aplicada agora a artefatos de sistema ao vivo.
- Como uma definição de tarefa agendada mapeia para T1053.005 e por que encontrar uma aqui é significativo: sua caçada do 4x04 (Task 11 naquele projeto) procurou explicitamente por evidência de tarefa agendada e não encontrou nenhuma no export do SIEM — então, se a memória mostra que uma existiu, essa é uma lacuna de detecção real que sua caçada não poderia ter fechado com os dados que tinha.

Como abordar

- Trabalhe seção por seção do arquivo, na ordem que a tarefa lista: processos, depois conexões de rede, depois módulos, depois o extrato de registro/tarefa agendada. Não tente fazer as quatro coisas em uma única passada.
- Para a triagem de processos, seu filtro não é só “o nome parece estranho” — cruze também o caminho (está rodando de um diretório padrão do sistema ou de `Temp/Public/ProgramData`, locais que projetos anteriores já sinalizaram como pontos de staging suspeitos?), e se está assinado digitalmente, caso essa informação esteja presente no artefato.
- Para conexões de rede, você tem duas coisas independentes para checar em cada uma: a infraestrutura de C2 conhecida no IOC master JSON, E especificamente o IP não identificado que o James Chen sinalizou — trate esse IP como sua própria mini-investigação, não só mais uma linha na tabela.
- Para a tarefa agendada, extraia todo campo que a saída esperada pede (nome, gatilho, ação/comando, timestamp de criação) como strings literais que você possa citar depois — a Task 7 vai precisar encaixar esse evento exato na linha do tempo maior.

Comandos e padrões para usar

```
# Cruzar um indicador candidato (nome de processo, IP, hash) contra o IOC master:
jq -e --arg val "VALOR_CANDIDATO" '.. | select(. == $val)?'
reference/healthbane_ioc_master.json
# (sai com 0 / imprime um match se KNOWN; não imprime nada se NEW — ajuste o
# caminho jq conforme a estrutura real dos arrays do IOC master, ex: .domains[], .ips[])

# Puxar tudo no arquivo de memória que parece uma linha de conexão de rede:
grep -A4 -iE 'connection|ESTAB|established' ir_evidence/memory_artifacts.txt
```

```
# Isolar só o bloco de tarefa agendada / registro para leitura atenta:  
grep -n -iE 'scheduled task|TaskCache|trigger|action' ir_evidence/memory_artifacts.txt
```

Fique de olho em

Um processo rodando de C:\Windows\System32\ com nome tipo svchost.exe não é automaticamente legítimo — é um alvo clássico de masquerading (você mesmo sinalizou esse risco, T1036, lá no mapeamento ATT&CK do 4x02). Confira o caminho completo e o processo-pai, não só o nome do arquivo, antes de rotular qualquer coisa como LEGITIMATE.

O que é este exercício

Você está processando `ir_evidence/disk_forensics_report.txt`, que captura *histórico* em vez de um único momento: arquivos deletados recuperados, registros de execução do Prefetch, o XML completo da tarefa agendada (confirmando a Task 1), persistência via chaves Run do registro, uma linha do tempo do \$MFT da NTFS para a janela do ataque, e indicadores de anti-forense. O achado de maior peso nesta tarefa são os arquivos deletados recuperados — a tarefa é explícita que confirmar o staging de dados muda isso de “incidente de segurança” para “provável vazamento de dados reportável.” Trate essa seção com o cuidado que ela merece.

O que você deve aprender

- A diferença prática entre **acesso** e **staging**: um atacante que apenas se conectou a um servidor de banco de dados é um achado bem diferente de um atacante que rodou consultas, escreveu os resultados em disco, e os comprimiu — e essa distinção tem consequências regulatórias diretas (veja a nota sobre HIPAA nos recursos do projeto).
- Como a evidência de Prefetch responde a uma pergunta que memória e rede não conseguem: um *histórico de execução* completo do que rodou neste host, com timestamps de primeira/última execução, mesmo para programas que não estão mais presentes no disco.
- Como raciocinar sobre anti-forense: uma lacuna curta e cirúrgica no log de eventos de Security é um achado bem diferente (e mais sofisticado) de um “Log Cleared” completo — a ausência do Event ID 1102 ao lado de uma pequena lacuna de timestamp é, em si, uma pista sobre o nível de sofisticação do atacante.

Como abordar

- Puxe primeiro a tabela de arquivos recuperados e raciocine sobre ela em três partes: padrão de nomeação (parece uma exportação de dados estruturada?), tamanho (o tamanho do .csv é consistente com saída de consulta, e os .zip parecem versões comprimidas dele?), e ordem cronológica (os dados foram puxados ANTES ou DEPOIS da compressão — isso te diz o fluxo de trabalho do staging).
- Para o Prefetch, seu filtro é baseado em função: WS-RECV-03 é uma estação de atendimento do departamento de Records. Confira `reference/network_topology.txt` (ou o contexto do cronograma de admin do 4x04 que você já conhece) para o que a pegada de software NORMAL desse host deveria ser, depois sinalize qualquer coisa na lista do Prefetch que não tem nada a ver com esse tipo de máquina.
- Monte a linha do tempo NTFS como seu próprio entregável — todo evento com timestamp que você achar em qualquer outro lugar deste relatório (criação de tarefa, criação de arquivo, deleção) pertence a uma única lista ordenada. Isso vira entrada direta para a reconstrução de movimento lateral/staging da Task 7, então acerte a ordenação agora.
- Para anti-forense, não reporte só “existe uma lacuna” — anote a duração dela e se o Event 1102 (limpeza total de log) está presente ou ausente, já que essa distinção é exatamente o que separa a deleção seletiva do T1070.001 de uma limpeza mais grosseira e barulhenta.

Comandos e padrões para usar

```
# Puxar só o bloco de arquivos deletados recuperados:
grep -A20 -i 'recovered.*deleted\|deleted files' ir_evidence/disk_forensics_report.txt

# Extrair linhas de Prefetch e ordenar pela primeira execução para ver o que rodou primeiro:
```

```
grep -iE '\.exe' ir_evidence/disk_forensics_report.txt | sort -t' ' -k2

# Montar sua própria linha do tempo estilo NTFS a partir de todo timestamp no arquivo:
grep -oE '[0-9]{4}-[0-9]{2}-[0-9]{2}([ T][0-9]{2}:[0-9]{2}(:[0-9]{2})?)?' \
ir_evidence/disk_forensics_report.txt | sort -u
```

Fique de olho em

Não confunda “staging aconteceu” com “exfiltração aconteceu.” Esta tarefa só tem evidência de disco — ela pode dizer que dados foram comprimidos e colocados num diretório de staging, e até que os arquivos de staging foram deletados depois. Mas, sozinha, ela não pode dizer se esses bytes algum dia saíram da rede. Essa pergunta específica é deliberadamente deixada para a Task 3 (análise de firewall), que é a única fonte de evidência capaz de realmente confirmar ou descartar exfiltração. Mantenha sua conclusão da Task 2 limitada ao que evidência de disco consegue provar.

O que é este exercício

ir_evidence/firewall_sessions_ws_recv_03.json é um array JSON de registros de sessão com estado (origem, destino, porta, protocolo, bytes transferidos, duração, timestamps) cobrindo a janela INTEIRA de 14 dias — a primeira fonte de evidência nesta reconstrução toda que faz isso. Sua cobertura de PCAP no 4x01 era de apenas 48 horas; este arquivo é a única evidência posicionada para responder definitivamente à pergunta sobre exfiltração, porque contagens de bytes em metadados de sessão são exatamente o que você precisa para ver dados saindo da rede, mesmo sem o conteúdo do pacote.

O que você deve aprender

- Por que metadados de log de sessão (bytes transferidos, duração) podem responder perguntas que capturas de pacote às vezes não conseguem, simplesmente por causa da janela de coleta — amplitude-de-tempo vs. profundidade-de-conteúdo é um trade-off real e recorrente em coleta de evidência.
- Como caracterizar um IP desconhecido de forma metódica: contagem de sessões, total de bytes, porta/protocolo, e especificamente seu PADRÃO DE TEMPO (intervalo fixo? só fora do horário?) — é literalmente a mesma análise de intervalo de beaconing que você já construiu para lógica de detecção no 4x01 e no 4x04, agora aplicada a um indicador novo.
- Como testar diretamente uma hipótese de exfiltração com aritmética: o total de bytes de saída suspeitos excede, bate aproximadamente, ou fica abaixo dos tamanhos dos arquivos de staging que você mediu na Task 2? Essa comparação É o achado de exfiltração — não é um julgamento qualitativo, é um número que você pode calcular.

Como abordar

- Comece pela divisão geral interno-vs-externo antes de correr atrás do IP desconhecido — você precisa da linha de base da população completa (quanto dado normalmente trafega, quantas sessões por dia) antes que os números de um IP específico signifiquem algo em contexto.
- Monte o perfil do IP desconhecido como seu próprio mini-relatório com todo campo que a saída esperada lista: primeira/última vez vista, contagem de sessões, padrão de intervalo, bytes de entrada/saída. Depois cruze explicitamente a porta e o protocolo contra o que o 4x01 documentou para o canal de C2 conhecido — semelhança (ou falta dela) é sua principal evidência para a decisão de C2-secundário-vs-tráfego-não-relacionado.
- Para a análise temporal, agrupe sessões por hora do dia nos 14 dias todos, depois puxe separadamente só as sessões EXTERNAS fora do horário e compare as datas delas com as datas específicas que sua caçada do 4x04 já sinalizou como noites de movimento lateral — alinhamento aqui é evidência corroborante forte.
- Deixe a comparação do limiar de exfiltração por último, depois que você tiver os dois números (bytes de saída suspeitos vs. tamanhos de staging da Task 2) bem calculados — não faça essa comparação “a olho”, calcule os dois lados com precisão e declare o delta.

Comandos e padrões para usar

```
# Contagens de sessão e bytes totais, interno vs externo:
jq '[.[] | select(.dest_ip | test("^10\\."))] | length'
ir_evidence/firewall_sessions_ws_recv_03.json
jq '[.[] | select(.dest_ip | test("^10\\.") | not)] | '\
'length=\(length) bytes=\(.[.].bytes_out | add)'\ \
ir_evidence/firewall_sessions_ws_recv_03.json
```

```
# Principais destinos externos por bytes totais (ajuste os nomes de campo ao seu arquivo):
jq -r '.[ ] | select(.dest_ip | test("^10\\.") | not) | '\
' [.dest_ip, .dest_port, .bytes_out] | @tsv' ir_evidence/firewall_sessions_ws_recv_03.json \
| awk '{sum[$1" "$2]+=$3} END {for (k in sum) print sum[k], k}' | sort -rn | head -10

# Aritmética de epoch-segundos para análise de intervalo (checagem de beaconing fixo):
jq -r --arg ip "IP_DESCONHECIDO" '.[ ] | select(.dest_ip==$ip) | .timestamp' \
ir_evidence/firewall_sessions_ws_recv_03.json | while read -r ts; do date -d "$ts" +%s; done |
sort -n
```

Fique de olho em

Confira os nomes reais dos campos no seu JSON de firewall antes de copiar qualquer padrão jq literalmente — os exemplos acima assumem nomes de campo plausíveis (dest_ip, bytes_out, timestamp); rode `jq '.[0]'` no arquivo primeiro para ver o schema real. Lembre também da lição do 4x04 sobre firewall vs. timestamps de captura de rede representarem momentos diferentes (início-de-conexão vs. captura-no-meio-da-sessão) — espere um delta pequeno e explicável ao cruzar os timestamps deste arquivo com os do 4x01, não um match exato.

O que é este exercício

Esta é a dobradiça analítica do projeto inteiro. Você pega tudo que as Tasks 0–3 produziram, mais todo resumo em `previous_findings/`, e monta três matrizes: uma matriz de correlação de IOC (quais fontes contêm quais indicadores), uma matriz de correlação de linha do tempo (quais fontes apoiam quais eventos-chave, e onde elas discordam), e uma matriz de correlação de técnica (quais técnicas ATT&CK são apoiadas por quais fontes, e se as chamadas INFERRED originais do 4x02 agora podem ser confirmadas ou precisam ser corrigidas). É aqui que cinco investigações separadas param de ser cinco investigações separadas.

O que você deve aprender

- Como classificar formalmente convergência de evidência em vez de descrevê-la em prosa: todo IOC único recebe exatamente um rótulo — CONVERGED, SINGLE-SOURCE, ou CONFLICTED — baseado estritamente em quantas fontes independentes o contêm.
- Como resolver uma contradição de timestamp corretamente: documente AMBOS os valores, proponha uma causa mecânica específica (fuso horário, desvio de relógio, atraso no ponto de coleta, atraso de agregação), e depois declare explicitamente qual fonte você trata como autoritativa daqui pra frente e por quê — “escolhi o número maior” não é uma resolução.
- Por que o status de uma técnica pode mudar em três direções diferentes quando mais evidência chega: INFERRED → CONFIRMED (nova evidência direta apareceu), permanece INFERRED (nenhuma evidência nova em nenhum sentido), ou — a que as pessoas esquecem que é possível — CORRECTED (a nova evidência mostra que a inferência original estava especificamente errada, não só não confirmada).

Como abordar

- Monte a matriz de IOC como uma tabela literal: uma linha por indicador único, uma coluna por fonte (4x00 até IR), marque presença. Não tente guardar isso na cabeça — um pipeline `jq/awk` no formato de planilha que gera exatamente essa tabela vai te salvar de perder uma convergência espalhada entre dois arquivos que você leu com uma hora de diferença.
- SÓ DEPOIS que a matriz de IOC estiver pronta, varra as linhas com 2+ marcas (sua lista CONVERGED) e linhas parecidas mas que não batem exatamente (seus candidatos a CONFLICTED — por exemplo, o mesmo IP de C2 relatado com uma porta ou hostname ligeiramente diferente em duas fontes).
- Para a matriz de linha do tempo, escolha suas âncoras temporais primeiro (o punhado de eventos em que você tem mais confiança, geralmente os com mais corroboração independente), depois trabalhe de dentro para fora, ordenando eventos menos certos relativos a essas âncoras em vez de tentar fixar cada evento num timestamp absoluto de forma independente.
- Para a matriz de técnica, percorra o mapeamento ATT&CK do 4x02 (`reference/attck_navigator_80pct.json`, representando o estado pós-4x04) técnica por técnica e pergunte, para cada uma: as Tasks 0–3 ou os achados anteriores adicionaram NOVA evidência direta para essa técnica específica? Essa é uma passada mecânica, não criativa — resista à tentação de promover uma técnica sem um artefato específico por trás da promoção.

Comandos e padrões para usar

```
# Checagem rápida via diff: quais IOCs estão nos dados novos de firewall do IR
# mas NÃO estão no IOC master existente (forma rápida de começar sua
# lista de "IOCs novos do IR"):
jq -r '.[].dest_ip' ir_evidence/firewall_sessions_ws_recv_03.json | sort -u > /tmp/fw_ips.txt
```

```
jq -r '.. | strings | select(test("^[0-9]{1,3}\\.{3}[0-9]{1,3}$"))' \
reference/healthbane_ioc_master.json | sort -u > /tmp/known_ips.txt
diff /tmp/known_ips.txt /tmp/fw_ips.txt # linhas com '>' são novas no IR

# Delta em epoch-segundos entre dois timestamps de duas fontes diferentes, para
# resolver uma contradição de linha do tempo:
echo $(( $(date -d "TIMESTAMP_FONTE_A" +%s) - $(date -d "TIMESTAMP_FONTE_B" +%s) ))
```

Fique de olho em

Um achado CONFLICTED é um resultado, não um fracasso — não descarte silenciosamente uma contradição só porque resolvê-la é mais trabalhoso do que reportar uma convergência limpa. Os requisitos do projeto são explícitos: contradições devem ser resolvidas, não ignoradas, e uma contradição não resolvida (ou escondida) é exatamente o tipo de coisa que mina a credibilidade de um relatório perante o board que o James Chen descreveu.

O que é este exercício

A primeira tarefa de escrita narrativa. Usando sua matriz de correlação da Task 4 como espinha dorsal evidencial, você reconstrói o Estágio 1 (entrega do phishing até o roubo de credenciais) e o Estágio 2 (estabelecimento do canal de C2) como um relato citado e cronológico — não um resumo dos relatórios do 4x00/4x01, mas uma síntese nova que traz a evidência de firewall que esses projetos anteriores nunca tiveram acesso.

O que você deve aprender

- Como escrever um parágrafo de reconstrução corretamente: evento → citação(ões) de evidência → técnica ATT&CK → nível de confiança, toda vez, sem exceções para afirmações “óbvias”.
- Como uma única fonte de evidência nova (logs de firewall de 14 dias) pode redatar um achado de um projeto anterior — especificamente, determinar se o IP de C2 secundário (da Task 3) já estava ativo durante este estágio inicial, ou só apareceu depois, muda a história sobre quando o atacante realmente implantou sua infraestrutura reserva.
- Como escrever um achado PROBABLE com honestidade: o canal de C2 secundário, por construção, só tem UMA fonte (os logs de firewall — a janela de PCAP do 4x01 terminou antes dele aparecer). É um PROBABLE de livro-texto, não CONFIRMED, e o exercício está explicitamente testando se você resiste à tentação de arredondar pra cima.

Como abordar

- Escreva o Estágio 1 primeiro, e trate-o como a metade mais fácil: o 4x00 já te dá a linha do tempo do phishing, os domínios, e os resultados de SPF/DKIM; seu trabalho aqui é principalmente selecionar os fatos citáveis exatos e anexar os IDs ATT&CK, não descobrir nada novo.
- Para o Estágio 2, comece pelo achado de intervalo de beacon do 4x01 como seu padrão de referência, depois vá às sessões de firewall e procure sessões para o MESMO IP de C2 que batam com esse intervalo — confirmar o padrão de forma independente é o que promove isso de fonte-única para CONVERGED.
- Resolva especificamente a questão de tempo do C2 secundário que a tarefa aponta: compare o timestamp de primeira aparição do IP desconhecido (da Task 3) contra sua linha do tempo do Estágio 2 — ele apareceu dentro da janela ou depois? Declare isso como um achado explícito com uma data, não uma impressão vaga.
- Atribua o nível de confiança enquanto escreve, não como um passo posterior — é bem mais fácil acertar isso enquanto a contagem de fontes para cada afirmação específica ainda está fresca na sua cabeça.

Comandos e padrões para usar

```
# Achar sessões de firewall para o IP de C2 conhecido e calcular intervalos entre
# sessões, para checar de forma independente o padrão de beacon de 5 minutos
# que o 4x01 já encontrou:
jq -r --arg ip "IP_C2" '.[.] | select(.dest_ip==$ip) | .timestamp' \
ir_evidence/firewall_sessions_ws_recv_03.json | sort | \
awk '{t=$0; "date -d \"\"t\"\" +%s" | getline e; if(p) print e-p; p=e}'

# Primeira aparição do IP desconhecido/secundário, para resolver a questão
# de "Estágio 2 ou depois":
jq -r --arg ip "IP_DESCONHECIDO" '.[.] | select(.dest_ip==$ip) | .timestamp' | sort | first' \
ir_evidence/firewall_sessions_ws_recv_03.json
```

Fique de olho em

Não deixe o Estágio 1 e o Estágio 2 se misturarem numa narrativa única e indiferenciada. A tarefa quer duas seções claramente separadas porque representam objetivos genuinamente diferentes do atacante (roubar uma credencial vs. estabelecer acesso remoto durável) e, muitas vezes, um intervalo de tempo real entre eles que vale a pena reportar como uma duração específica no seu resumo dos Estágios 1–2.

O que é este exercício

A tarefa narrativa culminante deste lote (Estágio 3 e Estágios 5/6, se existirem na sua lista completa de tarefas, não estão cobertos neste guia — veja a nota no final). O Estágio 4 é a fase de movimento lateral-e-staging que sua caçada do 4x04 descobriu comportamentalmente mas não conseguiu detalhar por completo. Aqui você funde QUATRO fontes de evidência — os achados da caçada do 4x04, memória do IR, disco do IR, e firewall do IR — numa única cadeia de pivots, e responde explicitamente às perguntas que a caçada sozinha deixou em aberto: qual credencial, quais hosts, o que exatamente foi acessado, e quão longe o staging foi antes da contenção.

O que você deve aprender

- Como montar uma cadeia de pivots como uma sequência literal de tuplas (host-origem → ferramenta → credencial → host-alvo → timestamp), cada uma citada individualmente, em vez de um parágrafo único descrevendo movimento lateral em termos gerais.
- Como evidência de IR pode revelar passos que uma caçada baseada em SIEM estruturalmente não conseguia ver — esta tarefa pede explicitamente que você procure passos de movimento lateral que a caçada PERDEU mas a evidência de memória/firewall revela, que é a ilustração mais clara do curso inteiro de por que caçada e IR são complementares, não redundantes.
- Como sequenciar corretamente uma pergunta de fluxo de dados (o atacante fez staging direto no WS-RECV-03, ou primeiro puxou dados dos servidores de banco de dados para o WS-RECV-03?) usando ordenação de timestamp entre fontes, não suposição — a ordem de criação de arquivos na linha do tempo NTFS da Task 2 é sua evidência principal aqui.
- Como escrever uma seção contrafactual (“se não tivesse sido contido”) com responsabilidade: ela precisa estar ancorada num número específico (tamanhos de arquivos de staging, cadência de sessões) e rotulada como uma estimativa, não apresentada como mais um achado confirmado.

Como abordar

- Monte a cadeia de pivots em ordem cronológica estrita, cruzando cada passo contra toda fonte que plausivelmente poderia tê-lo visto: uma execução de PsExec deveria aparecer na saída da caçada do 4x04 E, frequentemente, como uma sessão correspondente nos logs de firewall, e a credencial usada deveria remontar ao acesso à LSASS que você catalogou na Task 1.
- Para a subseção de acesso à credencial, não apenas repita o achado T1003.001 do 4x04 — o ponto inteiro desta tarefa é perguntar se a evidência de memória do IR mostra algo que a visão somente-SIEM da caçada não conseguia ver: um módulo carregado, um handle de processo, uma segunda credencial tocada que a caçada nunca teve visibilidade.
- Para acesso e staging de dados, cruze a linha do tempo NTFS da Task 2 diretamente com os timestamps da sua cadeia de pivots — a ordem de “arquivo de resultado de query criado” vs. “movimento lateral para servidor de banco de dados” vs. “arquivo comprimido criado” te diz definitivamente se os dados foram puxados PARA o host-pivot antes da compressão, que é exatamente a questão de sequenciamento que a tarefa pede para você resolver.
- Escreva a seção de contenção por último, depois que todo outro timestamp estiver fechado — o momento exato em que a detecção da caçada disparou precisa ser seu evento-confirmado mais recente, com tudo mais na cadeia vindo antes dele.

Comandos e padrões para usar

```
# Cruzar um timestamp específico da caçada do 4x04 contra dados de sessão
# de firewall para o mesmo host/porta, para achar sessões de movimento lateral
# corroborantes (ou perdas):
jq -r --arg host "HOST_ALVO" '[] | select(.dest_host==$host or .dest_ip=="IP_ALVO") | '\
' [.timestamp, .dest_port, .bytes_out, .bytes_in] | @tsv' \
ir_evidence/firewall_sessions_ws_recv_03.json | sort

# Ordenar todo evento de sistema de arquivos da Task 2 contra todo evento de sessão
# de rede da Task 3 numa única linha do tempo compartilhada (junte, depois ordene
# pelo primeiro campo):
cat task2_eventos_fs.tsv task3_sessoes.tsv | sort -k1,1

# Estimativa simples de duração para a seção "se não tivesse sido
# contido" (edite os números para sua própria evidência): bytes
# restantes / taxa média de transferência observada:
python3 -c "print(round(BYTES_RESTANTES / (BYTES_ATE_AGORA / SEGUNDOS_DECORRIDOS), 1))"
```

Fique de olho em

Esta tarefa pede explicitamente que você ache passos de movimento lateral que a caçada do 4x04 PERDEU. Não pule isso só porque parece estar questionando seu próprio trabalho anterior — uma caçada construída só sobre telemetria de SIEM/Sysmon tem pontos cegos estruturais (por exemplo, conexões que terminaram antes da coleta de log começar, ou hosts fora do escopo original da caçada), e achar uma dessas lacunas é um resultado genuinamente valioso, não uma crítica à caçada original.

O que é este exercício

Uma tarefa pura de fusão e verificação: pegue as listas de eventos que suas reconstruções dos Estágios 1–2 (Task 5), Estágio 3 (Task 6 — veja a nota abaixo), e Estágio 4 (Task 7) já produziram, e combine-as numa única sequência cronológica sem eventos duplicados, mais um conjunto de métricas temporais calculadas (dwell time, breakout time, tempo até persistência, tempo até staging, detecção-até-contenção). Esta tarefa não introduz evidência nova — é onde as três narrativas que você já escreveu são testadas uma contra a outra.

O que você deve aprender

- Por que uma linha do tempo unificada consegue revelar um erro que três narrativas separadas por estágio escondem: a tarefa dá o exemplo de malware parecendo ter sido implantado ANTES do C2 ser estabelecido, o que é logicamente impossível para essa cadeia de ataque — você só pega esse tipo de contradição quando tudo está num único eixo ordenado.
- Como transformar uma narrativa reconstruída em números reais: dwell time, breakout time, e cadência operacional não são julgamentos qualitativos, são aritmética sobre dois timestamps que você já tem — é a mesma habilidade de subtração em epoch-segundos do 4x04, só que aplicada à reconstrução inteira em vez de uma única caçada.
- Por que alguns eventos genuinamente não podem ser ordenados um em relação ao outro com confiança, e por que dizer isso explicitamente (em vez de escolher uma ordem arbitrária) é a escolha mais rigorosa.

Como abordar

- Extraia a lista de eventos de cada script de estágio como linhas estruturadas (você vai querer um formato de linha consistente — timestamp / descrição / técnica / confiança / fontes — nas Tasks 5, 6 e 7 especificamente para que este passo de fusão seja mecânico em vez de uma reescrita).
- Concatene e ordene por timestamp. Preste atenção especial na fronteira entre estágios: o último evento citado no final do Estágio 2 e o primeiro evento no início do Estágio 3 podem ser a MESMA evidência citada por dois ângulos — remova essa sobreposição em vez de listá-la duas vezes.
- Calcule cada métrica temporal como uma diferença direta de epoch-segundos entre dois eventos específicos e nomeados na sua própria linha do tempo (por exemplo, breakout time = timestamp do primeiro evento de movimento lateral menos timestamp do acesso inicial) — declare quais dois eventos você usou para cada métrica, para que o número seja reproduzível, não só afirmado.
- Faça a passada de identificação de lacunas por último: uma vez que a sequência completa esteja ordenada, procure qualquer período de horas/dias com zero eventos e pergunte se é um período dormiente real (consistente com a cadência operacional já conhecida do atacante) ou uma lacuna de coleta (nenhuma fonte de evidência tinha visibilidade naquela janela).

Comandos e padrões para usar

```
# Mesclar três listas de eventos já ordenadas e reordenar pelo campo de
# timestamp (coluna 1):
cat eventos_estagio1_2.tsv eventos_estagio3.tsv eventos_estagio4.tsv | sort -t'$'\t' -k1,1

# Calcular uma métrica temporal como um delta simples de epoch-segundos entre
# dois eventos nomeados:
T0=$(date -d "TIMESTAMP_PRIMEIRO_ACESSO" +%s)
```

```
T1=$(date -d "TIMESTAMP_PRIMEIRO_MOVIMENTO_LATERAL" +%s)
echo "Breakout time: $(( (T1 - T0) / 3600 )) horas"

# Achar lacunas: calcular o delta entre cada par CONSECUTIVO de eventos na lista
# mesclada e ordenada, e sinalizar qualquer delta acima de um limiar como candidato
# a lacuna:
awk -F'\t' '{cmd="date -d \"\"$1\"\" +%s"; cmd | getline e; close(cmd); if (p) print e-p, $1;
p=e}' timeline_mesclada.tsv
```

Fique de olho em

A Task 6 (Reconstrução do Estágio 3) é uma dependência direta e nomeada aqui (“Mescla as saídas de T5, T6 e T7”) mas não foi incluída no material fornecido para este guia. Pela estrutura do relatório na Task 15, o Estágio 3 é quase certamente “Implantação de Malware” — construída da mesma forma que as Tasks 5 e 7, usando os achados de malware do 4x03 mais a evidência de memória do IR confirmando que o processo do RAT/dropper estava realmente rodando. Você vai precisar das instruções reais dessa tarefa antes da Task 8 ficar completa; este guia pode ser estendido para cobri-la se você colar a descrição dela.

O que é este exercício

A reavaliação definitiva, em escala de projeto inteiro, de toda técnica ATT&CK no modelo de ameaça, usando `reference/attck_navigator_80pct.json` como seu ponto de partida. Diferente de todo trabalho ATT&CK anterior (que avaliou técnicas usando o que quer que uma única investigação tivesse acabado de produzir), esta tarefa reavalia CADA técnica contra o quadro COMPLETO de evidência que você agora tem das Tasks 0–8, depois adiciona cinco técnicas novas explicitamente nomeadas que a evidência de IR introduziu, e calcula a porcentagem final de cobertura.

O que você deve aprender

- Que a reavaliação de técnica não é uma catraca de mão única. A tarefa pede explicitamente que você considere resultados CORRECTED e DOWNGRADED, não só promoções — se a evidência de IR contradiz um achado CONFIRMED anterior, o movimento honesto é dizer isso, mesmo que pareça estar desfazendo trabalho anterior.
- Como uma única evidência nova pode resolver uma chamada INFERRED antiga: é a mesma lógica de promoção que você já usou uma vez no 4x04 (a Task 11 lá promoveu 5 técnicas de NOT COVERED para OBSERVED usando evidência da caçada) — aqui você está fazendo mais uma passada, informada pela evidência de IR que a caçada nunca teve.
- Por que 100% de cobertura não é o objetivo e deveria te deixar desconfiado se você chegar lá. A saída esperada deixa explicitamente 1 técnica como uma lacuna genuína — aprenda a distinguir “não temos evidência nem a favor nem contra” (uma limitação real, reportada com honestidade) de “estamos confiantes que isso não foi usado” (também um achado legítimo, mas diferente).

Como abordar

- Percorra o JSON de base técnica por técnica, não fonte-de-evidência por fonte-de-evidência — para cada técnica já presente, pergunte “algo nas Tasks 0–8 adicionou evidência a favor ou contra esta técnica específica?” antes de ir para a próxima. É uma passada mecânica; resista à tentação de promover em lote técnicas que apenas parecem relacionadas a coisas que você achou.
- Adicione as cinco técnicas novas explicitamente listadas (T1053.005, T1074.001, T1560.001, T1070.001, T1005) usando as mesmas citações de evidência que você já construiu nas Tasks 1 e 2 para cada uma — você já fez a análise subjacente, esta tarefa é principalmente sobre encaixá-la formalmente na estrutura ATT&CK (tática, confiança, quem identificou primeiro).
- Calcule a cobertura como uma fração direta (técnicas confirmadas-ou-melhor ÷ total de técnicas no modelo de ameaça) e mostre seu cálculo, da mesma forma que você já fez nas métricas da Task 8 — não só afirme uma porcentagem.
- Escreva a seção de “lacuna remanescente” com honestidade intelectual real: declare explicitamente se a lacuna é uma limitação de coleta (nenhuma fonte poderia tê-la visto) ou uma ausência genuína (você tem razão positiva para acreditar que a técnica não foi usada).

Comandos e padrões para usar

```
# Padrão reaproveitável do 4x04 para promover score/color/comment de uma
# técnica no lugar (ajuste a lista de IDs e os comentários para sua
# própria reavaliação):
jq --argjson ids '["T1053.005","T1074.001","T1560.001","T1070.001","T1005"]' '
```

```
.techniques += [$ids[] | {techniqueID: ., score: 3, color: "#c40000", enabled: true}]  
' reference/attck_navigator_80pct.json > attack_layer_final.json
```

```
# Calcular a porcentagem final de cobertura direto do array de técnicas:  
jq ' [.techniques[] | select(.score==3)] | length' attack_layer_final.json  
jq '.techniques | length' attack_layer_final.json
```

Fique de olho em

Antes de confiar em qualquer porcentagem de base do resumo armazenado de um projeto anterior, recalcule-a diretamente a partir do array de técnicas daquele arquivo. Essa precaução exata salvou trabalho real no projeto 4x04 (o resumo armazenado do próprio arquivo ATT&CK de referência dizia “16 observed” mas isso não batia com uma contagem direta dos scores das técnicas, que na verdade era 14) — sempre verifique um número de base antes de construir em cima dele, em vez de confiar num rótulo.

O que é este exercício

A pergunta de negócio que tudo o mais neste projeto existe para responder: dados de pacientes foram comprometidos, e isso cruza o limiar de notificação de vazamento da HIPAA? Você mapeia todo host comprometido contra `reference/meddefense_asset_inventory.txt` para ver que dado ele guarda, classifica o nível de acesso de cada host (confirmado / potencial / sem acesso), resolve a questão da exfiltração comparando os totais de staging da Task 2 contra os totais de saída suspeita da Task 3, e combina os dois numa tabela de exposição por tipo de dado com uma determinação regulatória.

O que você deve aprender

- Por que “dado foi acessado” e “dado foi exfiltrado” são achados legalmente distintos com consequências de notificação diferentes — e por que uma reconstrução que confunde os dois não é só impreciso, pode ter consequência legal real.
- Como aplicar um modelo de acesso de três níveis (CONFIRMED / POTENTIAL / NO ACCESS) a sistemas em vez de afirmações individuais — CONFIRMED precisa de um artefato específico de interação (por exemplo, um timestamp de criação de arquivo batendo com a janela do atacante), POTENTIAL é alcance-mais-credenciais sem evidência de interação, e confundir os dois é exatamente o erro que torna uma avaliação de vazamento indefensável.
- Como uma comparação de contagem de bytes que você já meio-construiu na Task 3 vira o fato decisivo para toda a questão de exfiltração — esta tarefa não pede que você re-derive esse número, pede que você o use como a dobradiça de uma conclusão legal/regulatória.

Como abordar

- Monte primeiro o mapeamento host-para-dado, como uma busca simples: pegue todo host que apareceu em qualquer lugar da sua cadeia de movimento lateral da Task 7 e cruze contra a classificação de sensibilidade de dados do inventário de ativos. Esta parte é mecânica.
- Resolva o nível de acesso por host usando uma regra evidencial estrita, não impressão: você citou um artefato específico na Task 7 mostrando que o atacante interagiu com dados neste host (confirmado), ou só que ele tinha a credencial e o caminho de rede para alcançá-lo (potencial)? Hosts que nunca estiveram na cadeia de movimento lateral são simplesmente sem acesso.
- Resolva a questão de exfiltração reaproveitando a própria comparação da Task 3 (bytes de saída suspeitos vs. tamanhos de staging da Task 2) — não recalcule do zero, cite. Declare explicitamente se a comparação mostra que houve transmissão, foi parcial, ou não aconteceu, e POR QUÊ (o que interrompeu, com um timestamp).
- Só depois que nível de acesso e status de exfiltração estiverem ambos resolvidos, monte a tabela por tipo de dado (PHI / seguro / funcionários / operacional) — o status de cada linha deve seguir direta e rastreavelmente dos achados em nível de host acima dela, não ser atribuído de forma independente.
- Escreva a seção regulatória como um silogismo direto: aqui está a evidência, aqui está a linguagem do limiar da HIPAA, aqui está se a evidência atinge esse limiar, aqui estão os fatores atenuantes (tempo de interrupção, qualquer criptografia, controles de acesso) que pesam na severidade mesmo que o limiar seja atingido.

Comandos e padrões para usar

```
# Cruzar sua lista de hosts de movimento lateral contra a classificação de
# dados do inventário de ativos:
grep -F -f hosts_comprometidos.txt reference/meddefense_asset_inventory.txt
```

```
# Reaproveitar (não recalcular) a comparação de exfiltração da Task 3:
BYTES_STAGING=364000000 # da Task 2, em bytes
BYTES_SAIDA=$(jq '[.] | select(.dest_ip=="IP_SUSPEITO") | .bytes_out] | add' \
ir_evidence/firewall_sessions_ws_recv_03.json)
echo "Staging: $BYTES_STAGING Transmitido para destino suspeito: ${BYTES_SAIDA:-0}"
```

Fique de olho em

Resista a arredondar “acesso potencial” para “confirmado” só porque parece mais conclusivo, e resista a arredondar “confirmado” para “potencial” para parecer conservador. As duas direções são desonestas da mesma forma: as audiências regulatória e do board, explicitamente nomeadas neste projeto, precisam da linha evidencial real, não de uma versão suavizada ou inflada — esse é todo o ponto de construir um modelo de três níveis em vez de um simples sim/não.

O que é este exercício

O entregável final — um relatório em Markdown, não um script — que sintetiza toda tarefa de T0 até T14 num único documento servindo duas audiências ao mesmo tempo: o time de SOC (detalhe técnico, mapeamentos ATT&CK, especificações de detecção) e o board (narrativa clara, impacto, cronograma de remediação). É a mesma disciplina de “duas audiências” que todo relatório capstone deste curso já pediu, agora aplicada na escala da campanha inteira em vez de uma única investigação.

O que você deve aprender

- Como escrever um documento de síntese que não re-deriva análise — toda seção deste relatório deve remontar a uma tarefa que você já completou e comprimir o achado dela em prosa de relatório, não refazer o trabalho inline.
- Como escrever um resumo executivo que resiste a escrutínio: um parágrafo cobrindo o que aconteceu, quão longe o atacante chegou, como foi parado, o que vem a seguir, e as métricas principais — denso, mas toda frase rastreável até a saída de uma tarefa específica.
- Por que a seção de conclusões pede para você refletir sobre a própria tese do curso (“por que investigar em pedaços cria pontos cegos que só a reconstrução revela”) — esse é o momento de articular, com suas próprias palavras, o que esse arco de seis projetos inteiro estava ensinando.

Como abordar

- Construa esta tarefa POR ÚLTIMO, depois que a saída de toda outra tarefa genuinamente existir — ela é explicitamente uma síntese de T0 até T14, e várias de suas seções obrigatórias (referência do layer ATT&CK Navigator de T10, Postura Defensiva de T13, Plano de Remediação de T14) dependem de tarefas não cobertas em nenhum lugar deste guia ainda.
- Percorra a lista de seções obrigatórias do relatório como um checklist contra suas próprias tarefas completas: Resumo Executivo, Metodologia (de T0), Reconstrução do Ataque (T5–T8), Linha do Tempo Unificada (T8), Análise ATT&CK (T9–T11), Avaliação de Impacto (T12), Postura Defensiva (T13), Plano de Remediação (T14), Conclusões, Apêndices. Se a tarefa-fonte de uma seção ainda não existe, essa seção ainda não pode ser escrita com honestidade.
- Para os apêndices (tabela resumo de IOC, índice de citações de evidência, referência do layer ATT&CK), esses são quase exportações diretas de trabalho que você já construiu — a matriz de correlação de IOC da Task 4 e o inventário de técnicas da Task 9 são seu material-fonte principal, reformatado para um apêndice em vez de re-analisado.
- Escreva o resumo executivo POR ÚLTIMO, mesmo que ele apareça primeiro no documento — é muito mais fácil comprimir um relatório já completo num único parágrafo apertado do que escrever o resumo primeiro e depois fazer o resto do relatório bater com ele.

Fique de olho em

Esta tarefa tem dependências reais e nomeadas que ainda não estão cobertas neste guia: Task 6 (Estágio 3), Task 10 (provavelmente o próprio arquivo do layer ATT&CK Navigator, referenciado como apêndice), Task 11 (não fica claro pelo contexto — possivelmente uma análise de lacunas/pontos-cegos, já que a Task 15 cita uma “Análise de lacunas e avaliação de pontos cegos” sob Análise ATT&CK), Task 13 (Avaliação da Postura Defensiva), e Task 14 (Plano de Remediação). Não comece a Task 15 antes de ter as instruções e saídas reais

dessas cinco tarefas — me envie e este guia pode ser estendido antes de você precisar delas.

Uma Nota Sobre o Que Não Está Neste Guia

Este guia cobre as Tasks 0, 1, 2, 3, 4, 5, 7, 8, 9, 12 e 15 — onze tarefas, todas confirmadas como o conjunto completo de material fornecido quando este guia foi construído. Cinco números de tarefa são lacunas conhecidas, cada uma uma dependência confirmada e nomeada de uma tarefa coberta aqui, não um chute:

Task 6 (Reconstrução do Estágio 3 — dependência direta da fusão da Task 8 e quase certamente “Implantação de Malware”, inferido da estrutura do relatório da Task 15), **Task 10** (arquivo do layer ATT&CK Navigator, referenciado como apêndice da Task 15), **Task 11** (provavelmente uma análise de lacunas/pontos-cegos, referenciada na seção “Análise ATT&CK” da Task 15), **Task 13** (Avaliação da Postura Defensiva), e **Task 14** (Plano de Remediação). Se você tiver essas cinco descrições de tarefa, me envie e este guia pode ser estendido para cobrir o projeto completo.

Um último conselho que se aplica a toda tarefa acima: construa suas citações de evidência à medida que avança, não no final. Quando você chegar na Task 7 ou na Task 8, você estará puxando de seis ou mais saídas anteriores ao mesmo tempo, e o hábito de escrever “[arquivo-fonte / campo / timestamp]” ao lado de cada fato no momento em que você a encontra — em vez de tentar reconstruir de onde um fato veio depois — é a coisa isolada mais provável de te poupar tempo no projeto inteiro.